

CS126 Warwick+ Report

5685010 - Szymon Galutowski

1 Data Structures Created

In total, I implemented 3 generic data structures: **MyHashTable** (a hashmap), **MyAVLTree** (an AVL tree), and **MyLinkedList** (a singly linked list). The hashmap is used in all three stores, as they all include a certain object where $\mathcal{O}(1)$ retrieval time is either necessary or possible. The AVL tree is only used in stores where they benefit from the sorting power of the tree, such as Movies and Ratings. The linked list is used almost everywhere, where having a dynamic data structure for quick retrieval of multiple items is necessary. Since all of these data structures are generic, it means I can use them anywhere in the program, save hours of time, and keep redundancy and clutter to a minimum.

1.1 Hashmap

The reason I use a hashmap for storing most of the data is because $\mathcal{O}(1)$ retrieval time is the ideal user experience. However, retrieval time does not always remain $\mathcal{O}(1)$. As collisions occur, the inserted values must be stored somewhere. I opted to use chaining to resolve these collisions. To aid with this **MyHashTable** class defines a separate class within, called **HashNode**. Each **HashNode** contains a pointer to the next node in the chain, acting like a singly linked list. They also store the object itself.

To combat increasing load factor as the size of the hashmap increases, I implemented a **resize()** function. Once the hashmap reaches a load factor of 0.75 or above, the **resize()** method is triggered, ensuring retrieval remains close to $\mathcal{O}(1)$ at all times.

1.2 AVL Tree

The defining feature of an AVL tree is its self-balancing properties. Since it is essentially a binary search tree, the self-balancing property allows, for each node, a maximum height deviance of ± 1 between the right and left subtrees. Because of this strict balancing, the maximum height of the entire tree is locked at approximately $1.44 \log_2 n$. This ensures the tree doesn't degrade into a linked list, maintaining its $\mathcal{O}(\log n)$ retrieval time. The AVL tree is also optimised for range queries, where values with keys within a given range are retrieved. The time complexity for this operation is $\mathcal{O}(\log n + k)$. This is done by only traversing down branches of the tree that have keys that fall within the range provided.

In my case, **MyAVLTree** stores **AVLNode**, defined within the class structure. Each **AVLNode** stores its key, its values (as a linked list), height, and pointers to the left and right children. This implementation allows for multiple values to be stored at each key, as is the case in Movies and Ratings, where to ensure efficiency, multiple data objects share a key.

1.3 Linked List

For short, unimportant stores everywhere in the database, I use my own singly linked list, called **MyLinkedList**. Each element of the list is its own object, called **MyListElement**, which

stores the value, and a pointer to the next element. The list itself stores the head, and amount of elements. This simple structure was much better than an `ArrayList`, since those require resizing every time a new element is added. With an $\mathcal{O}(1)$ insertion time and worst case $\mathcal{O}(n)$ retrieval time, it's simple, and I often use it for storing a list of values within data store objects where the list may change size.

2 Algorithms Used - Quicksort

The only external algorithm I implemented was the quicksort algorithm. I chose this due to its $\mathcal{O}(n \log n)$ average case time complexity. Additionally, it maintains efficiency when sorting arrays with $> 1\text{M}$ values. I implemented it using the same style as what was taught in the lecture notes - i.e. using Lomuto-style partitions. While the time complexity may degrade into $\mathcal{O}(n^2)$ worst case, I opted to use the median of three strategy to make the best case more viable.

I use a helper function to find the pivot index. This is to reduce clutter, as I want to use a series of if statements. The alternative is to use `Math.min()` and `Math.max()` to create a one liner, but this causes increased overhead due to external function calls. To ensure the quicksort method is generic, I pass in a lambda function that defines the type of the values, and how they will be compared.

3 The Movies Class Architecture

For the Movies store, I maintain two hashmaps and an AVL tree:.

- `moviesTable` - This stores every single `MovieNode`, using the `tmdb_id` as the key.
- `collectionTable` - This stores every single `CollectionNode`, using the `collection_id` as the key.
- `movieTree` - This stores every single `movieID` as long as it has a non-null release date, using release as the key.

The `MovieNode` and `CollectionNode` objects store all the necessary information about the movie and collection respectively. This ensures that the `moviesTable` is the primary method of retrieving any information about a movie, as all you need is its ID.

The AVL Tree (`movieTree`) is used to store the `movieID` as the value, and release date as the key. This allows each movie to be sorted in the tree by release date. When populating the database, occasionally the release date would be null. In order to avoid Null Pointer Exceptions, I only stored the movie ID in the tree if the release date wasn't null. This tree is only used for one particular function call - that is, `getAllIDsReleasedInRange()`. This function requests a list of movie IDs where each movie's release date is within the given range. Using the original `MovieNode` hashmap for this would cause the retrieval time to be $\mathcal{O}(n)$. However, by using the tree, I'm able to reduce the time down to $\mathcal{O}(\log n + k)$.

4 The Credits Class Architecture

For the Credits store, I maintain three hashmaps:.

- `creditsTable` - This stores every single `CreditNode`, using `credit_id` as the key.
- `castTable` - This stores every single cast member as a `PersonNode`, using `person_id` retrieved from the `Person` object as the key.

- **crewTable** - This stores every single crew member as a **PersonNode**, using **person_id** retrieved from the **Person** object as the key.

Each **CreditNode** object stores all the credit information for a given movie, and each **PersonNode** object stores information about a specific cast or crew member's credits for a given person.

All of these hashmaps are essentially storing people, but they can all be retrieved by different means. This means that if I need information about a cast or crew given a specific movie, I check the **creditsTable**. However, if I need to check against every single cast/crew member, such as in **getUniqueCast/Crew()**, then I check the **cast/crewTable**. This allows for $\mathcal{O}(1)$ retrieval for anything concerning a specific known person or movie. Multiple functions in Credits require the results to arrive sorted - this is where the implemented quicksort comes in.

5 The Ratings Class Architecture

For the Ratings store, I maintain two hashmaps and three AVL trees:.

- **userTable** - This stores every single **UserRatingNode**, using **userID** as the key.
- **movieTable** - This stores every single **MovieRatingNode**, using **movieID** as the key.
- **userTree** - Stores every single **userID**, using the amount of ratings they've made as the key.
- **movieTree** - Stores every single **movieID**, using the amount of ratings it has as the key.
- **averageTree** - Stores every single **movieID**, using its average rating as the key.

Each **UserRatingNode** stores their ID, all the ratings they've ever made, as well as the amount and average rating. The **MovieRatingNode** stores their ID, all the ratings for the movie, as well as the amount and average rating. Each contains methods to retrieve **Rating** objects from their list of **Rating** objects using the ID of the other, as there is no single rating ID. Each **Rating** object stores the **userID**, the **movieID**, the rating value, and the timestamp.

These two tables fulfill the requirement of being able to lookup ratings by both **userID** and **movieID**. There is also a requirement for being able to retrieve user IDs and movie IDs by the amount of ratings they've made, and the average rating. That is what the trees are for, as their sorted structure allows for $\mathcal{O}(\log n)$ lookup time.

6 Efficiency

6.1 Theoretical Efficiency

The time complexity of data structures represents search, insertion, and deletion time.

Hashmap

Variation	Time Complexity	Space Complexity
Best Case	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Average Case	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Worst Case	$\mathcal{O}(n)$	$\mathcal{O}(n)$

Variation	Time Complexity	Space Complexity
Best Case	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$
Average Case	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$
Worst Case	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$

Variation	Time Complexity	Space Complexity
Best Case	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Average Case	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Worst Case	$\mathcal{O}(n)$	$\mathcal{O}(n)$

AVL Tree

Linked Lists

Quicksort

Variation	Time Complexity	Space Complexity
Best Case	$\mathcal{O}(n \log n)$	$\mathcal{O}(\log n)$
Average Case	$\mathcal{O}(n \log n)$	$\mathcal{O}(\log n)$
Worst Case	$\mathcal{O}(n^2)$	$\mathcal{O}(n)$

6.2 Actual Timing Results

For each class, I have listed relevant methods and their average execution time given random parameters, based on the 1000 movie dataset provided. Explanation of what each function does is in the JavaDoc comments.

Movies Method	Average Time
<code>getAllIDs()</code>	0.170ms
<code>getAllIDsReleasedInRange()</code>	0.0168ms
<code>findFilms()</code>	3.26ms

Credits Method	Average Time
<code>getUniqueCast()</code>	4.21ms
<code>getUniqueCrew()</code>	1.42ms
<code>getMostCastCredits()</code>	221ms (outlier!)
<code>findCast()</code>	8.22ms
<code>findCrew()</code>	5.25ms

Ratings Method	Average Time
<code>getTopAverageRatedMovies()</code>	0.0475ms
<code>getMostRatedMovies()</code>	0.0613ms
<code>getMostRatedUsers()</code>	0.0325ms

7 Evaluation

Overall, I'm satisfied with the database. Everything runs reasonably quickly, compared to the UI rendering logic which causes a huge bottleneck. However, if I had more time, I would've added an AVL tree to the Credits class to store `PersonIDs` by the amount of credits they have, for cast. This would reduce my `getMostCastCredits()` methods down to $\mathcal{O}(\log n + k)$ time complexity, rather than $\mathcal{O}(n \log n)$.